

STARVNT

UNIVERSAL VENDOR OPERATING SYSTEM

Developer Master Architecture & Implementation Specification

Location + Capability + Portfolio + Commercial + Execution + Intelligence

Purpose: give the development team one locked architecture for Vendor OS. Category-specific workspaces may differ, but business truth, commercial execution, automation and core state remain shared.

What this document locks

A single Vendor OS model, Service Location as the primary geographic anchor, capability-aware matching, total-cost validation, execution evidence, and Core authority.

What this document rejects

Separate category backends, category-specific booking engines, vendor-declared payment truth, duplicate automation, nearest/lowest-quote shortcuts, and UI-only implementations.

1. EXECUTIVE ARCHITECTURE CONTRACT

Customer tells STARVNT what they are planning. STARVNT determines what services are required. Vendors declare what they can deliver, where, when, with what capacity, at what commercial terms, and with what evidence.

The Vendor OS is not a directory. It is the operating layer that turns a vendor business into structured, matchable, bookable, executable supply inside STARVNT.

Master lifecycle

```
CUSTOMER -> EVENT -> REQUIREMENTS -> SERVICE LOCATION -> VENDOR CAPABILITY
-> ELIGIBILITY -> AVAILABILITY -> CAPACITY -> OPERATIONAL FEASIBILITY
-> TRAVEL/LOGISTICS -> VALIDATED TOTAL COST -> SUITABILITY -> OPPORTUNITY
-> ENQUIRY -> QUOTE -> APPROVAL -> BOOKING -> PAYMENT -> EXECUTION
-> COMPLETION SUBMITTED -> CORE VALIDATION -> SETTLEMENT -> PERFORMANCE -> AURA+
```

Core product rule

One Vendor OS. One commercial engine. One automation spine. Category-specific capability and workspace only.

Developer outcome: the same architecture must support photography, videography, makeup, venue, catering, decoration, DJ/production, transport, hotel, bridal wear, inventory vendors and future categories without creating parallel domains.

Definition of Done

A feature is not complete because the screen exists. It is complete when the underlying business state is persisted in the correct owner, the correct event/automation path fires, authorization is enforced, duplicate execution is safe, failure is recoverable, and the working path can be demonstrated end-to-end.

2. SYSTEM BOUNDARY & OWNERSHIP

The following separation must remain explicit in code, APIs, database ownership and frontend behavior.

Layer	Primary responsibility	Authority	Do not duplicate
Core Platform	Identity, org/tenant, event, requirement, booking, payment truth, completion validation, settlement state, audit	Core / Keshab	No second booking/payment/event authority
Vendor OS	Vendor identity, services, capabilities, coverage, availability, capacity, resources, pricing, packages, opportunities, vendor execution inputs	Vendor domain / Yogesh	No second payment or settlement authority
Central Automation	Trigger, condition, action, worker, retry, idempotency, recovery, notifications, orchestration	Automation / Nesar	No parallel automation engine
Revenue CRM + Aura+	Lead lifecycle, intelligence context, recommendations, prioritization, growth insight	Revenue/Aura+ / Sourav	No duplicate Core business truth
Frontend	Human interaction, guided workflows, vendor actions, evidence submission	UI layer	Frontend is not a security boundary

Critical rule

Aura+ recommends. Automation executes authorized actions. Core decides and records authoritative business state.

Example: vendor can submit completion evidence; vendor cannot declare COMPLETION_VERIFIED, PAYMENT_VERIFIED or SETTLEMENT_ELIGIBLE.

3. UNIVERSAL VENDOR MASTER DATA MODEL

```
VENDOR
IDENTITY + OWNERSHIP + VERIFICATION
SERVICES
CAPABILITIES
CAPACITY | COVERAGE | AVAILABILITY | PRICING | PACKAGES | TERMS
OPERATING LOCATIONS
RESOURCES
TEAM | EQUIPMENT | INVENTORY | VEHICLES | SPACES
PORTFOLIO / MEDIA / PROOF
OPPORTUNITIES -> ENQUIRIES -> QUOTES -> BOOKINGS
EXECUTION -> COMPLETION -> PAYMENTS -> REVIEWS -> ANALYTICS -> AURA+
```

Vendor activation state

```
DRAFT -> REGISTERED -> PROFILE_INCOMPLETE -> VERIFICATION_PENDING -> VERIFIED -> ELIGIBLE -> ACTIVE
```

ACTIVE is not a profile checkbox. A vendor/service becomes commercially active only when the required service model is sufficiently complete: capability, coverage, availability, pricing and required verification.

Service model contract

Object	Minimum responsibility
Vendor	Who the business is; organization ownership; contacts; verification state
Service	What the vendor sells/fulfills
Capability	How the service can be delivered; style/type/scope; category-specific attributes
Capacity	People, team, equipment, inventory, vehicles, space, daily/simultaneous/geographic capacity
Coverage	Where this exact service/capability can be fulfilled
Availability	When the required resources are actually available
Pricing	Base + unit + travel + logistics + conditional charges + tax/commercial terms
Portfolio	Evidence of work, media, event type, location, style, service, verification
Execution Profile	What must be done, by whom, where, when and how completion is evidenced

4. LOCATION-CENTRIC ARCHITECTURE

Location is a business capability dimension, not an address field. The matching engine must never collapse all geography into one vendor address.

```
CUSTOMER LOCATION = contextual origin / user context
EVENT DESTINATION = broad event geography
VENUE LOCATION = exact property
SERVICE LOCATION = where a specific requirement must be fulfilled
VENDOR BASE LOCATION = operating origin
EXECUTION LOCATION = actual fulfillment point
```

Service Location is the primary geographic anchor for vendor matching.

A single event can contain many service locations. Example: bride home for makeup, Kisan Palace for photography/catering/decor, airport for transport, hotel for accommodation.

ServiceLocation should support at minimum: address, coordinates, city, locality, venue, space, access instructions, contact, service time, setup time, teardown time, parking, restrictions and location type.

Vendor operating locations

A vendor can have multiple operational origins: head office, studio, branch, warehouse, kitchen, workshop, equipment hub, storage. Coverage is separate from these origins.

Coverage belongs to Service/Capability

Photography may cover Kolkata + radius; decoration may cover the state; catering may be city-only; DJ may allow outstation. Coverage must be evaluated against the specific requirement, not merely the vendor record.

5. ADVANCED LOCATION & OPERATIONS INTELLIGENCE

Dimension	Required behavior
Coverage type	Locality, city, district, state, radius, route, destination, venue, airport, hotel, special service area
Coverage confidence	SELF_DECLARED -> VERIFIED -> SYSTEM_OBSERVED -> BOOKING_PROVEN
Preference vs eligibility	Preference may influence ranking; eligibility remains a hard gate
Location-specific capability	Different team/resource capacity may apply by service area
Travel/logistics	Distance, mode, people, equipment, toll, parking, accommodation, allowance and other verified cost inputs
Time + geography	Availability must incorporate travel time, setup windows, team and equipment
Multi-location job	Validate route, travel buffer, resource continuity and execution sequence
Geographic conflict	A calendar that looks free may still be operationally impossible because of travel
Location risk	Remote venue, access restrictions, overnight stay, traffic dependency, tight setup window, weather-sensitive work
Location performance	Store service-level performance by locality/venue/route when evidence exists

True Availability

```
DATE + TIME + LOCATION + TEAM + EQUIPMENT + CAPACITY = OPERATIONAL FEASIBILITY
```

The system should be able to surface: "Calendar says available, but resource or travel constraints make this opportunity infeasible."

6. CAPACITY & RESOURCE MODEL

Do not implement capacity as a single integer. Capacity is category-specific and may be geographic, resource-based and simultaneous.

Category	Examples of capacity/resources
Photography / Videography	Photographers, videographers, teams, cameras, equipment, simultaneous events, event duration
Catering	Guests, kitchen throughput, staff, meals/day, service format, menu production
Decoration	Area, inventory, setup team, simultaneous event count, transport/warehouse stock
Venue	Space, seated capacity, standing capacity, parking, facilities, booking slot
Transport	Vehicles, passengers, luggage, drivers, trips, route/time
Hotel	Room inventory, room types, occupancy, property availability
Bridal wear / inventory	Item-level stock, item reservation, size/color availability, dispatch/return timing
DJ / Production	Sound system, LED, lighting, stage, truss, genre fit, technical inventory, crew

Resource conflict examples

A photographer may show availability but have the only camera team allocated elsewhere. A decorator may have a free calendar but insufficient inventory for two simultaneous events. A transport vendor may have vehicles but no feasible driver/route combination. These must be modeled as operational constraints, not manual assumptions.

7. PORTFOLIO & MEDIA AS A FIRST-CLASS DOMAIN

The portfolio is evidence of capability, not decoration. Vendor onboarding must support bulk upload and structured organization.

```
BULK UPLOAD -> PROCESSING -> TYPE DETECTION -> CLASSIFICATION
-> TAG SUGGESTION -> THUMBNAIL/TRANSCODING -> VENDOR REVIEW -> PUBLISH
-> VERIFICATION -> OPTIONAL BOOKING-PROVEN EVIDENCE
```

Unified Photography + Videography workspace

A vendor can upload photos, videos, albums, films, reels, teasers, highlight films and full-event samples in one operation. The system should support service, category, style, event type, location, venue, date, deliverable type, duration, featured status, visibility and verification state.

Portfolio item lifecycle

```
UPLOADED -> PROCESSING -> VENDOR_REVIEW -> PUBLISHED -> VERIFIED -> (OPTIONAL) BOOKING_PROVEN
```

Aura+ may assist with organization and classification. The vendor approves. Core stores the structured portfolio truth.

Location-linked evidence

Example: wedding photography at Kisan Palace, New Town. Future opportunity ranking can use proven work at similar service/location/venue combinations as a suitability signal.

8. COMMERCIAL ENGINE: LOWEST VALIDATED TOTAL COST

STARVNT must not optimize for nearest vendor or lowest visible quote. The objective is the lowest validated total cost among genuinely eligible vendors.

```
REQUIREMENT -> SERVICE LOCATION -> COVERAGE -> HARD ELIGIBILITY
-> AVAILABILITY -> CAPACITY -> VENUE COMPATIBILITY
-> OPERATIONAL FEASIBILITY -> TRAVEL/LOGISTICS
-> VALIDATED TOTAL COST -> QUALITY/RELIABILITY -> RANKING
```

Total cost model

```
BASE VENDOR PRICE
+ TRAVEL
+ EQUIPMENT TRANSPORT
+ SETUP / DELIVERY
+ TOLL / PARKING
+ ACCOMMODATION / ALLOWANCES
+ OTHER VERIFIED CHARGES
= VALIDATED TOTAL COST
```

Hard eligibility must be deterministic: category/service match, date availability, capacity, coverage, active/eligible state, requirement applicability, constraints and venue compatibility where relevant.

Soft ranking may consider total cost, quality, reliability, location performance, availability confidence, customer preference and operational risk.

9. OPPORTUNITY -> ENQUIRY -> QUOTE

Vendor should not receive an unqualified raw customer message when STARVNT already has enough context to create a structured opportunity.

```
REQUIREMENT -> SERVICE LOCATION -> SERVICE -> CAPABILITY -> COVERAGE
-> AVAILABILITY -> CAPACITY -> FEASIBILITY -> COST -> SUITABILITY
-> VALIDATED OPPORTUNITY -> ENQUIRY
```

Opportunity example

Field	Example
Service	Wedding Photography
Event date	26 Nov
Service location	Kisan Palace, New Town
Guest count	500
Required capability	2 photographers; Candid + Traditional; full day
Coverage	Vendor serves New Town
Availability	Eligible slot found
Estimated travel	Barasat -> New Town
Travel cost	■2,000 estimated/validated as policy allows
Action	Respond / Quote

Quote lifecycle

```
DRAFT -> SUBMITTED -> APPROVED / REJECTED / EXPIRED
```

Invalid transitions must be blocked server-side, with a clear error state and audit. Quote approval may create BOOKING_REQUEST through Central Automation, but Booking remains Core authority.

10. CATEGORY WORKSPACES - SHARED ENGINE, DIFFERENT CAPABILITY

Do not create separate backends. Category-specific workspace modules should configure capability and UX on top of the same domain model.

Category	Capability examples	Critical operational logic
Photographer / Videographer	Styles, teams, cameras, packages, deliverables, drone	Team/resource allocation, venue/date coverage, multi-location schedule, portfolio evidence
Makeup	Bridal, HD, airbrush, hair, draping, assistants, client slots	Client slot + travel + location + simultaneous client limits
Venue	Spaces, seated/standing, facilities, parking	Space-level inventory, booking period, venue compatibility
Caterer	Guests, cuisine, menu, live counters, dietary, staff	Guest quantity + kitchen + menu + service logistics
Decorator	Stage, mandap, floral, themes, lighting, furniture	Area + inventory + setup/dismantle team + access/power
DJ / Production	Sound, DJ, LED, lighting, stage, genre, crew	Venue technical compatibility + equipment logistics
Transport	Vehicle, passenger, luggage, route, trips	Origin -> destination + driver/vehicle + route/time feasibility
Hotel	Property, room types, inventory, occupancy	Room inventory availability + property/location match
Inventory / Bridal wear	Items, variants, stock, reservation	Actual item availability, dispatch/return windows

11. EXECUTION, COMPLETION & SETTLEMENT

Payment success is not settlement success. Completion is a separate service lifecycle and must be validated by Core/Policy.

```
BOOKING CONFIRMED
-> PAYMENT VERIFIED
-> SERVICE SCHEDULED
-> SERVICE STARTED
-> MILESTONE / SERVICE DELIVERY
-> COMPLETION SUBMITTED
-> CORE VALIDATION
-> COMPLETION VERIFIED
-> SETTLEMENT ELIGIBLE
-> SETTLED
```

Vendor execution workspace should expose the event/service location, customer, team, timing, requirements, instructions, equipment/resources and category-specific completion checklist.

Completion examples

Service	Completion evidence examples
Photography	Coverage completed + agreed deliverables/proof submission
Catering	Meal/service completion against agreed guest/service scope
Decoration	Setup delivered, event operation completed, dismantling where required
Transport	Pickup/route/drop/trip completion
Makeup	Agreed client service delivered
Venue	Booked period/service delivered as contracted

Non-negotiable

Vendor frontend must never be the authority for PAYMENT_VERIFIED, COMPLETION_VERIFIED, SETTLEMENT_ELIGIBLE or SETTLED. Vendor submits facts/evidence. Core validates and records business truth.

12. AUTOMATION SPINE & EVENT CONTRACTS

Every Vendor OS action that crosses domain boundaries should use the existing Central Automation spine rather than introducing another orchestration implementation.

```
DOMAIN EVENT -> OUTBOX -> WORKER -> TRIGGER -> CONDITION -> AUTHORIZED ACTION
-> SUCCESS / RETRY -> ACTIVITY + AUDIT
```

Representative flow

```
EVENT_CREATED
-> EVENT_TYPE_RESOLVED
-> REQUIREMENT_CREATED
-> VENDOR_MATCHED
-> VENDOR_ENQUIRY_CREATED
-> QUOTE_CREATED
-> QUOTE_SUBMITTED
-> QUOTE_APPROVED
-> BOOKING_REQUEST
-> BOOKING_CONFIRMED
```

Execution / settlement orchestration

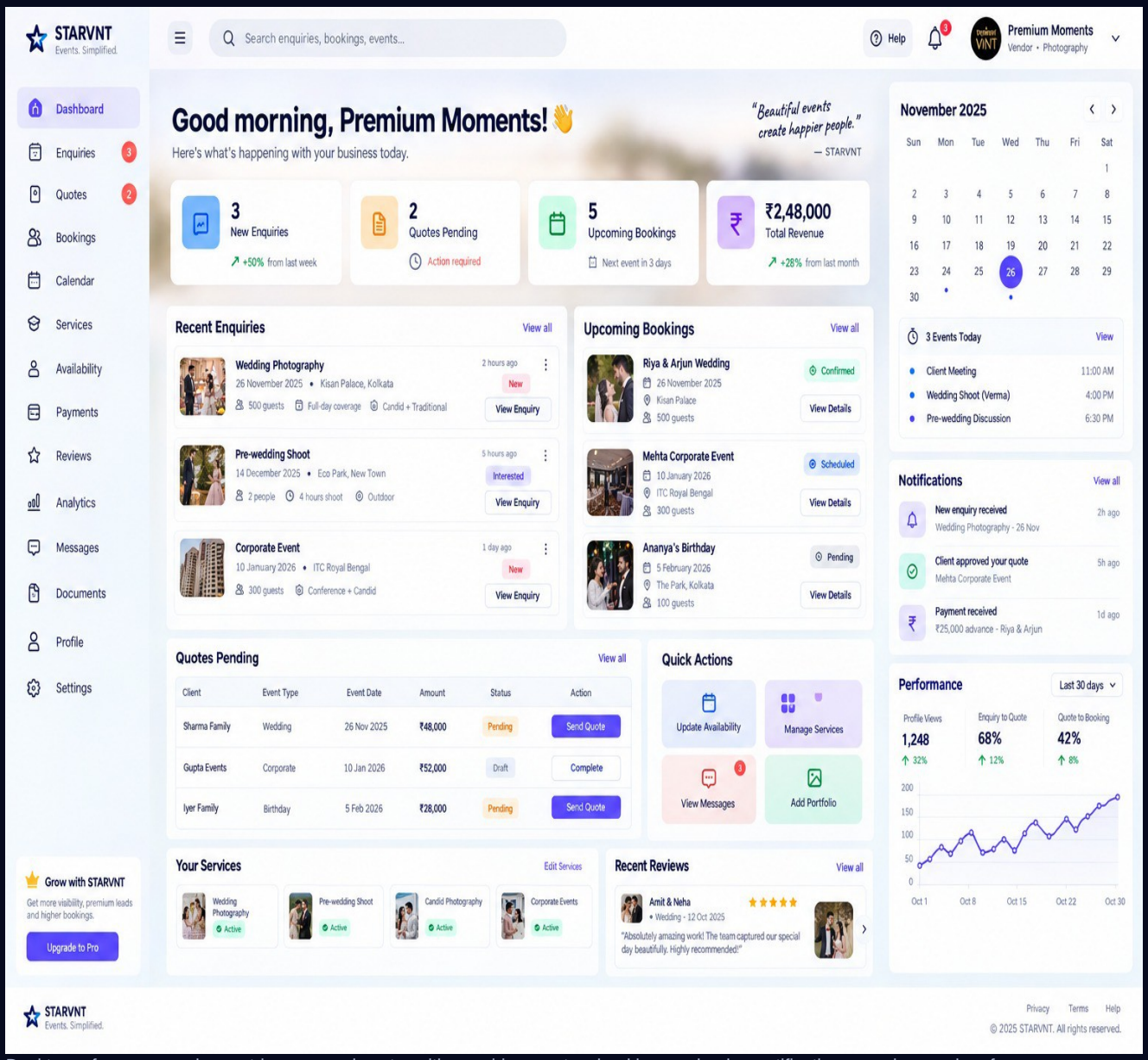
```
PAYMENT_CONFIRMED -> SERVICE_SCHEDULED -> SERVICE_STARTED
-> MILESTONE -> COMPLETION_SUBMITTED -> COMPLETION_VERIFIED
-> SETTLEMENT_ELIGIBLE -> SETTLED
```

Reliability requirements

Duplicate events, duplicate webhooks, concurrent actions, worker failures and downstream failures must be idempotent and recoverable. Retry must not produce duplicate business outcomes. All meaningful transitions must be auditable.

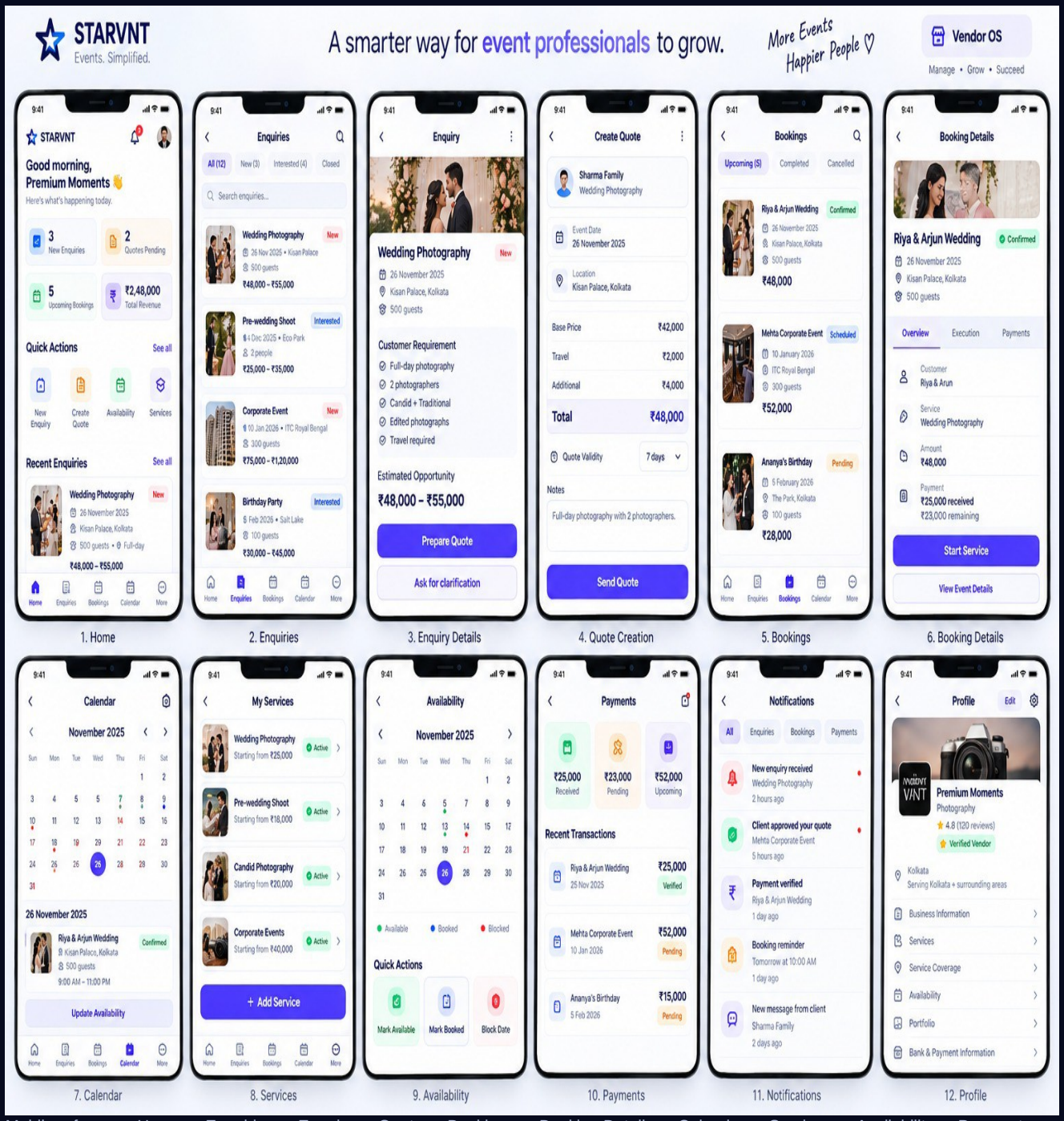
13. VENDOR OS UX - REFERENCE EXPERIENCE

The supplied UI references establish direction, not a pixel-perfect requirement. The frontend should expose the domain model clearly and keep actions aligned with backend authority.



Desktop reference: vendor-centric command center with enquiries, quotes, bookings, calendar, notifications, services and performance.

13A. MOBILE VENDOR WORKSPACE



Mobile reference: Home -> Enquiries -> Enquiry -> Quote -> Bookings -> Booking Details -> Calendar -> Services -> Availability -> Payments -> Notifications -> Profile. The final implementation should preserve the same business contracts as desktop.

14. VENDOR OS INFORMATION ARCHITECTURE

```
HOME
OPPORTUNITIES
ENQUIRIES
QUOTES
BOOKINGS
CALENDAR

MY BUSINESS: Profile | Services | Capabilities | Packages | Pricing | Availability
LOCATIONS: Operating Locations | Service Coverage | Service Areas | Travel | Logistics
PORTFOLIO & MEDIA: Photos | Videos | Albums | Films | Reels | Categories | Tags | Verification
RESOURCES: Team | Equipment | Inventory | Vehicles | Spaces
EXECUTION | COMPLETION | PAYMENTS | MESSAGES | REVIEWS | ANALYTICS | AURA+ | PROFILE
```

Home should answer: "What opportunities and actions do I have today?"

Key dashboard objects: new opportunities by location, quotes awaiting response, upcoming bookings, payment outcomes, geographic conflicts, notifications, current services, performance and actionable recommendations.

Customer, Vendor, Aura+ and Core questions

Surface	Primary question
Customer	How is my event progressing?
Vendor OS	What opportunities and actions do I have today?
Aura+	What should I know or do next, and why?
Core	What is officially true?

15. CUSTOMER -> STARVNT -> VENDOR BEHAVIOR

The customer should not be forced to think in vendor-management terminology. STARVNT should derive structured requirements from ordinary language and progressively clarify only what is necessary.

Example input

"My daughter's wedding is on 26 November, 500 guests, New Town, and I want good photography, catering and decoration within budget."

System responsibility

```
UNDERSTAND EVENT
-> IDENTIFY EVENT TYPE / DATE / GUESTS / VENUE
-> DISCOVER SERVICE NEEDS
-> RESOLVE SERVICE LOCATIONS
-> APPLY REQUIREMENT TEMPLATES
-> FIND MISSING / DEPENDENT SERVICES
-> DISCOVER ELIGIBLE VENDORS
-> COST + FEASIBILITY + SUITABILITY
-> PRESENT OPTIONS / RECOMMENDATIONS
-> APPROVAL -> BOOKING -> PAYMENT -> EXECUTION
```

Customer may explicitly request photography, catering and decoration while STARVNT may infer that makeup, sound, transport, accommodation, invitations, stage/lighting or hospitality may also be relevant. The system should assist without interrogating the customer with an unnecessary 15-question form.

Product principle

Customer tells STARVNT what they want. STARVNT figures out what it takes to make it happen.

16. AURA+ & VENDOR GROWTH INTELLIGENCE

Aura+ sits above deterministic domain systems. It should not become a second CRM, pricing authority, booking engine or settlement system.

Signal	Possible Aura+ recommendation
Location demand	"You received repeated enquiries from Rajarhat; consider expanding verified service coverage."
Location performance	"New Town is currently your strongest service area by booking performance."
Opportunity quality	"This opportunity matches your service, coverage, capacity and proven venue experience."
Risk	"High logistics dependency: long travel + tight setup window + multiple service locations."
Resource conflict	"Your calendar is open, but your only available camera team is already allocated."
Portfolio gap	"Your portfolio contains many wedding photos but limited cinematic video evidence for this service."

Operating principle

Aura+ understands and recommends. Vendor approves where a business-setting change is required. Core stores the resulting truth.

17. SECURITY, TENANCY & DATA INTEGRITY

Vendor OS must inherit the existing Core security model. UI visibility alone is never sufficient.

Control	Acceptance expectation
Authentication	Unauthenticated domain actions rejected server-side
Authorization	Role/permission checked on every protected mutation
Tenant isolation	Vendor/organization cannot read or mutate another tenant's data
Ownership	Forged vendorId/orgId fields cannot override server-resolved context
State integrity	Invalid transitions rejected server-side
Idempotency	Duplicate event/webhook/action produces one business outcome
Audit	Important profile, service, coverage, quote, booking, execution, completion and settlement actions traceable
Core authority	Frontend cannot declare payment/completion/settlement truth
Evidence integrity	Portfolio/completion evidence includes ownership, visibility and verification metadata

Sensitive KYC/bank information should use secure onboarding and controlled storage; do not treat casual messaging fields as the system of record.

18. DEVELOPER IMPLEMENTATION CHECKLIST

Build order should follow domain authority and existing architecture. Do not create a new module merely because a screen needs data.

Step	Build / verify	Status gate
1	Review existing Core models/contracts before changing Vendor OS	Reuse identified
2	Vendor -> Service -> Capability -> Capacity model	Persisted + tested
3	Operating Locations + Service Coverage + Travel Policy	Persisted + queryable
4	Availability + Resource + Geographic conflict checks	Deterministic result
5	Pricing + travel/logistics + total-cost calculator	Repeatable calculation
6	Opportunity qualification + matching	Eligible candidates only
7	Enquiry + Quote lifecycle	Server-state machine
8	Quote approval -> Core Booking Request	Idempotent orchestration
9	Execution + category completion evidence	Core validation path
10	Portfolio/media bulk upload + structure	Processing + review + publish
11	Notifications + audit + retry + idempotency	Central Automation only
12	Vendor dashboard / mobile surfaces	Consumes real backend state
13	Aura+ recommendations	Reads domain truth; no duplicate authority
14	E2E proof	Working demo + DB/event/audit evidence

Mandatory engineering behavior

REVIEW existing code -> REUSE existing domain -> EXTEND contracts -> INTEGRATE -> TEST -> PROVE -> COMMIT.

19. GOLDEN E2E ACCEPTANCE TESTS

The Vendor OS is accepted only when the following scenarios are demonstrated against real running code.

Test	Expected proof
A. Vendor onboarding	Vendor reaches ACTIVE only when required service model is complete/eligible
B. Service coverage	Requirement location resolves against capability-level coverage, not vendor address only
C. Availability	Time + location + team + resource conflict correctly blocks infeasible booking
D. Total cost	Base + travel + logistics + verified additional cost produces deterministic total
E. Lowest validated cost	Lower-cost ineligible vendor is excluded; lowest eligible validated option is selected
F. Opportunity	Structured requirement becomes vendor opportunity with actionable context
G. Quote	Invalid quote transition returns controlled failure; valid submission persists
H. Approval	Duplicate approval does not create duplicate booking request
I. Payment	Payment verified state comes from Payment Core / server verification path
J. Execution	Vendor can submit service progress/evidence but cannot self-verify completion
K. Settlement	Settlement eligibility occurs only after completion validation
L. Failure recovery	Vendor failure can trigger eligible alternative discovery on the same requirement/location
M. Portfolio	50+ photos + multiple videos can be bulk uploaded, organized, reviewed and published
N. Security	Cross-tenant/vendor access fails; forged IDs ignored; audit trail present
O. Automation	Failure -> retry -> success without duplicate business outcome

Acceptance definition

COMPLETE & PROVEN = live E2E + persisted state + event/automation trace + authorization + failure/retry proof + audit.

20. FINAL ARCHITECTURAL LOCK

The following rules are the design constraints for every future Vendor OS feature.

Rule	Locked principle
1	Location is a business capability dimension, not an address field.
2	Service Location is the primary geographic anchor for matching.
3	Coverage belongs to Service/Capability.
4	Availability = Time + Location + Team + Resource.
5	Capacity is category-specific and can be geographic.
6	Travel and logistics are part of validated total cost.
7	Venue compatibility is separate from geographic coverage.
8	One Event can have many Service Locations.
9	One Vendor can have many operating locations and many services.
10	Nearest Vendor is not automatically the best vendor.
11	Lowest Quote is not automatically the lowest total cost.
12	Portfolio is evidence of capability.
13	Vendor submits execution/completion; Core validates authoritative truth.
14	Aura+ recommends; Core decides and records.
15	No category-specific backend/business truth unless a genuine domain boundary is approved.
16	No duplicate Booking, Payment, Event, Requirement or Automation engines.

ULTIMATE PRODUCT STATEMENT

STARVNT = Customer Planning Platform + Universal Vendor Operating System + Location Intelligence + Commercial Optimization + Execution Network + Aura+ Intelligence.

A vendor does not merely list on STARVNT. The vendor represents the full operating reality of the business: who they are, what they sell, where they operate, when they can serve, what resources they control, what they charge, what proof they have, what work they are executing and what outcomes they can reliably deliver.

Developer handoff note

Treat this document as the master direction for Vendor OS implementation. Resolve conflicts by checking Core authority and existing contracts first. Do not silently create a parallel flow to satisfy a local UI requirement.